



Yazılım Proje Yönetim Planı (SPMP)

Çok Ajanlı Sözleşme İnceleme ve Müzakere Orkestratörü
(Multi-Agent Legal Ops)

Grup İsmi: Lagent
Hazırlayanlar: Mustafa Emir Ceran - 24118080702
Mert Ayrancı - 24118080752
Osman Gazi Atalay - 23118080703
Tarih: 11 Mart 2026

İçindekiler

1	GİRİŞ	2
1.1	Proje Genel Bakışı	2
1.2	Proje Teslimatları	2
2	PROJE ORGANİZASYONU	2
2.1	Yazılım Süreç Modeli	2
2.2	Roller ve Sorumluluklar	2
2.3	Araçlar ve Teknikler	2
3	PROJE YÖNETİM PLANI	3
3.1	Görevler	3
3.1.1	Görev-1: Veri Alımı ve Madde Sınıflandırma	3
3.1.2	Görev-2: Politika Kontrolü ve Risk Değerlendirmesi	3
3.1.3	Görev-3: Revizyon ve Onay Akışı	4
3.1.4	Görev-4: Arayüz Geliştirme	4
3.2	Atamalar	5
3.3	Zaman Çizelgesi	6
4	EK MATERYALLER	7
5	KAYNAKÇA	7

1 GİRİŞ

1.1 Proje Genel Bakışı

Bu proje, bireylerin günlük yaşamlarında karşılaştıkları sözleşme ve hukuki metinleri daha anlaşılır, hızlı ve güvenli biçimde inceleyebilmesini amaçlamaktadır. Proje kapsamında geliştirilecek sistem; metinleri madde bazında ayrıştıran, kullanıcının tercih ettiği kurallara göre risk analizi yapan, alternatif (fallback) ifade önerileri sunan ve gerekli aksiyon adımlarını yöneten çok ajanlı (multi-agent) [25] bir yapay zeka asistanıdır. Sistem, gereksiz avukat süreçlerini elimine etmek için geliştirilecektir; bu sayede hem zaman hem de maddi açıdan tasarruf sağlamayı hedeflemektedir.

1.2 Proje Teslimatları

- **Kaynak Kod Deposu:** Tüm ajanların, RAG [24] altyapısının ve arayüzün bulunduğu, versiyonlanmış ve dokümente edilmiş kod tabanı.
- **Çalıştırılabilir Sistem:** Kullanıcıların giriş yapıp sözleşme yükleyebileceği, çalışan Web Dashboard.
- **Dokümantasyon:** SPMP,SRS,SDD,STD dokümanlarının paylaşılması.

2 PROJE ORGANİZASYONU

2.1 Yazılım Süreç Modeli

Projede kanban [1] yaklaşımı kullanılacaktır. İş akışı, kanban panosu üzerinde (Yapılacak, Devam Eden, Doğrulama, Tamamlandı) sütunlarıyla takip edilecek ve work-in-progress (WIP) limitleri uygulanacaktır. Agile [2] prensipleri doğrultusunda haftalık planlama/gözden geçirme toplantıları yürütülecek, iş kalemleri iki haftalık iterasyon hedefleriyle önceliklendirilerek analiz, tasarım, geliştirme ve test faaliyetleri paralel ilerletilecektir.

2.2 Roller ve Sorumluluklar

- **Mustafa Emir Ceran:** Kanban [1] akış yönetimi (pano, WIP limitleri, blokaj takibi), AI/LLM [26] ajanlarının gereksinim ve prompt stratejisinin yönlendirilmesi, paydaş iletişimi ve teslimat koordinasyonu.
- **Mert Ayrancı:** Backend ve API geliştirme, RAG [24]/veri tabanı entegrasyonu, risk ve politika ajanlarının teknik uygulanması, performans ve güvenlik iyileştirmeleri.
- **Osman Gazi Atalay:** Frontend ve redline/onay arayüzlerinin geliştirilmesi, uçtan uca test ve kalite kontrol süreçleri.

2.3 Araçlar ve Teknikler

- **Proje Yönetimi ve İş Takibi:** Kanban panosu yönetimi, iş atamaları, blokaj takibi ve WIP limitlerinin uygulanması için Trello [3] kullanılacaktır.
- **Versiyon Kontrolü ve CI/CD:** Kaynak kodun versiyonlanması ve ekip içi kod incelemeleri için GitHub [4] kullanılacaktır. Sürekli Entegrasyon ve Dağıtım (CI/CD) [27] süreçleri GitHub Actions [5] ile otomatikleştirilecek, ortam bağımsızlığı için Docker [6] tabanlı konteynerizasyon uygulanacaktır.

- **Geliştirme Altyapısı – Backend ve Yapay Zeka:** Python [7] tabanlı mikroservis [28] mimarisi (FastAPI [8]) üzerine kurulacaktır. Çok ajanlı [25] yapı ve LLM [26] entegrasyonları, durum tabanlı orkestrasyon araçları (LangGraph [9]) ve harici dil modelleri (OpenAI API [10]) ile sağlanacaktır.
- **Geliştirme Altyapısı – Frontend:** Kullanıcı etkileşimi ve arayüzler için React [12]/Next.js [13] tabanlı modern web teknolojileri kullanılacaktır.
- **Geliştirme Altyapısı – Veri Yönetimi:** İlişkisel veriler için PostgreSQL [14], RAG [24] altyapısında vektörel aramalar için pgvector [15], geçici önbellekleme/durum yönetimi için Redis [16] ve dosya depolama (Object Storage) için MinIO [17] tercih edilecektir.
- **Kalite Güvencesi ve Test:** Kod kalitesini korumak amacıyla birim ve entegrasyon testleri için Python tabanlı test framework'leri (Örn: pytest [18]), kullanıcı arayüzü ve uçtan uca testler için tarayıcı otomasyon araçları (Örn: Playwright [19]) kullanılacaktır.

3 PROJE YÖNETİM PLANI

3.1 Görevler

3.1.1 Görev-1: Veri Alımı ve Madde Sınıflandırma

- **3.1.1.1 Açıklama:** Yüklenen sözleşmelerin (PDF/Docx) metne dönüştürülmesi ve "Clause Agent" tarafından gizlilik, tazminat, fesih gibi kategorilere ayrıştırılması.
- **3.1.1.2 Teslimatlar:** Doküman parse eden API, sınıflandırılmış sözleşme maddesi (JSON formatında) veri seti.
- **3.1.1.3 Gerekli Kaynaklar:** PyMuPDF [20] + python-docx [21] ile doküman işleme altyapısı, OCR [22] servisi, sınıflandırma prompt şablonları, etiketli örnek sözleşme veri seti ve PostgreSQL [14]/MinIO [17] depolama kaynakları.
- **3.1.1.4 Bağımlılıklar ve Kısıtlar:** Güvenilir bir OCR kütüphanesinin entegre edilmesi.
- **3.1.1.5 Riskler ve Önlemler:** Karmaşık tablolu PDF'lerde parse hatası.
Acil Durum Planı: Gelişmiş Vision LLM'leri (Örn: GPT-4o [11]) veya AWS Textract [23] kullanımı.

3.1.2 Görev-2: Politika Kontrolü ve Risk Değerlendirmesi

- **3.1.2.1 Açıklama:** Kategorize edilen maddelerin, RAG [24] üzerinden "Kullanıcı Playbook'u" ile karşılaştırılması ve "Risk Agent" tarafından ticari/hukuki risk değerlendirilmesinin yapılması.
- **3.1.2.2 Teslimatlar:** Risk Değerlendirme Modülü, Karşılaştırma Raporu (Eksik veya çelişkili hükümlerin tespiti).
- **3.1.2.3 Gerekli Kaynaklar:** Kullanıcı playbook/politika dokümanları, pgvector [15] üzerinde indekslenmiş referans clause havuzu, risk değerlendirme rubriği ve kural motoru, OpenAI API [10] erişimi ve doğrulama için veri seti.
- **3.1.2.4 Bağımlılıklar ve Kısıtlar:** Görev-1'in tamamlanması.
- **3.1.2.5 Riskler ve Önlemler:** Modelin halüsinasyon görmesi ve risksiz bir maddeyi riskli işaretlemesi (False Positive).
Acil Durum Planı: LLM [26] "temperature" değerinin düşük tutulması ve kural motoru (if/else) ile LLM yanıtlarının çapraz doğrulanması.

3.1.3 Görev-3: Revizyon ve Onay Akışı

- **3.1.3.1 Açıklama:** Görev-2'de riskli olarak işaretlenen maddeler için, OpenAI API [10] ile clause-bazlı revizyon önerileri üretilecektir. Üretim sırasında kullanıcı Playbook'u, pgvector [15] üzerinde tutulan fallback clause kütüphanesi ve daha önce onaylanmış örnekler bağlam olarak kullanılacaktır. LangGraph [9] akışı; **Taslak Oluşturuldu → Hukuk İncelemesinde → İş Birimi Onayında → Onaylandı/Reddedildi** durum geçişleriyle yönetilecek ve kullanıcıya redline görünümü üzerinden madde bazlı karar verme imkanı sunulacaktır.
- **3.1.3.2 Teslimatlar:**
 - Revizyon API katmanı (FastAPI [8]) ve clause-bazlı öneri/karar uç noktaları.
 - Redline üretim servisi ve karşılaştırmalı çıktı setleri.
 - Onay akış motoru ve PostgreSQL [14] üzerinde audit trail kayıtları.
 - Gerekmesi halinde python-docx [21] ile revize sözleşme taslağı çıktısı.
 - Onay/red kararlarını takip eden durum raporu ekranı entegrasyonu.
- **3.1.3.3 Gerekli Kaynaklar:** Fallback clause kütüphanesi, onaylı revizyon örnekleri, Structured Output/Function Calling şemaları, LangGraph [9] akışları, PostgreSQL [14] audit kayıtları ve Redis [16] geçici durum yönetimi.
- **3.1.3.4 Bağımlılıklar ve Kısıtlar:** Görev-1 (maddeleme) ve Görev-2 (risk değerlendirme) çıktılarının tamamlanmış olması zorunludur. Kullanıcı Playbook'u ve fallback kütüphanesinin güncel sürümle yayımlanması, RAG [24] indekslerinin doğrulanması, Redis [16] tabanlı kısa ömürlü durum yönetiminin stabil çalışması ve yüksek riskli maddelerde insan onayı olmadan otomatik kabul yapılmaması temel kısıtlardır.
- **3.1.3.5 Riskler ve Önlemler:** LLM'in hukuki terminolojiyi bozması ve onay sıralarında darboğaz oluşması başlıca risklerdir.
Acil Durum Planı: Zorunlu insan onayı eşikleri ile çözüm.

3.1.4 Görev-4: Arayüz Geliştirme

- **3.1.4.1 Açıklama:** Kullanıcıların sözleşmeyi yükleyebileceği, işaretlenmiş maddeleri, risk değerlendirmesini ve revizyon önerilerini görebileceği web arayüzü.
- **3.1.4.2 Teslimatlar:** React [12] tabanlı web uygulaması.
- **3.1.4.3 Gerekli Kaynaklar:** Next.js [13]/React [12] geliştirme ortamı.
- **3.1.4.4 Bağımlılıklar:** Tüm Backend API'lerinin hazır
- **3.1.4.5 Riskler ve Önlemler:** Kullanıcı deneyiminin karmaşık olması.
Acil Durum Planı: Tutorial videoları eklenerek kullanım kolaylığı sağlamak veya kullanıcılara eğitim verilmesi.

3.2 Atamalar

Rol	Sorumlu Kiři	Sorumluluk Kapsamı
Proje Yönetimi ve AI Koordinasyonu	Mustafa Emir Ceran	- Kanban akış yönetimi ve blokaj takibi. - AI/LLM ajan gereksinimleri ile prompt stratejisinin yönetimi. - Negotiation/Redlining akış tasarımı. - Paydaş iletişimi ve final teslimat takibi.
Backend ve RAG Geliştirme	Mert Ayrancı	- Ingestion/OCR ve parse API geliřtirmesi. - RAG/veri tabanı entegrasyonu. - Risk/Policy agent teknik uygulaması. - Approval logic ve revizyon API geliřtirmesi. - Kritik backend hata ve performans iyileřtirmeleri.
Frontend ve Test/QA	Osman Gazi Atalay	- UI geliřtirme ve entegrasyon. - Risk çıktıları için arayüz gereksinimlerinin hazırlanması. - Temel doğrulama testleri, UAT ve uçtan uca test yürütümü.

3.3 Zaman Çizelgesi

Tarih Aralığı	Planlanan Çalışmalar
02 Mart–11 Mart 2026	- Proje başlangıcı, kapsam-plan netleştirme, genel hatlarıyla iş paketlerinin ve Kanban panosunun oluşturulması. - SPMP dokümanının hazırlanması; ingestion/OCR modeli ve mimari başlangıç kararlarının netleştirilmesi.
12 Mart–27 Mart 2026	- Gereksinim analizi ve kullanım senaryoları geliştirme başlangıcı; SRS'in ilk sürümünün oluşturulması. - Görev-1 kapsamında ingestion/parsing + OCR [22] altyapısının geliştirilmesi ve ilk sınıflandırma akışının prototiplenmesi. - Backend unit test iskeletinin (pytest [18]) ve frontend test altyapısının (Playwright [19]) kurulması.
28 Mart–10 Nisan 2026	- Sistem mimarisi, veri modeli ve API tasarımının detaylandırılması; SDD'nin ilk sürümünün teknik gözden geçirmeyle güncellenmesi. - Görev-2 kapsamında politika kontrolü, RAG [24] entegrasyonu ve risk skorelama modülünün geliştirilmesi; entegrasyon testlerinin başlatılması. - Frontend tarafında temel dashboard akışlarının geliştirilmesi ve backend ile erken entegrasyon testleri.
11 Nisan–24 Nisan 2026	- Görev-3 (redline/revizyon ve onay akışı) ve Görev-4 (arayüz) geliştirmelerinin devamı. - Uçtan uca senaryoların (Playwright [19]) genişletilmesi, kritik API akışlarında regresyon ve performans testlerinin yürütülmesi.
25 Nisan–08 Mayıs 2026	- Sistem stabilizasyonu, kullanıcı kabul testleri (UAT), güvenlik/perf iyileştirmeleri ve sürüm adayının hazırlanması.
09 Mayıs–13 Mayıs 2026	- STD dokümanının teslimi. - Proje final sunumunun gerçekleştirilmesi ve kapanış raporlaması.

4 EK MATERYALLER

5 KAYNAKÇA

Kaynaklar

- [1] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*, Blue Hole Press, 2010.
<https://www.amazon.com/Kanban-Successful-Evolutionary-Technology-Business/dp/0984521402>
- [2] K. Beck et al., *Manifesto for Agile Software Development*, 2001.
<https://agilemanifesto.org/>
- [3] Atlassian, *Trello – Manage Your Team's Projects From Anywhere*, 2024.
<https://trello.com/>
- [4] GitHub, Inc., *GitHub: Let's build from here*, 2024.
<https://github.com/>
- [5] GitHub, Inc., *GitHub Actions – Automate your workflow*, 2024.
<https://docs.github.com/en/actions>
- [6] Docker, Inc., *Docker: Accelerated Container Application Development*, 2024.
<https://www.docker.com/>
- [7] Python Software Foundation, *Python Programming Language*, 2024.
<https://www.python.org/>
- [8] S. Ramírez, *FastAPI – Modern, Fast Web Framework for Building APIs with Python*, 2024.
<https://fastapi.tiangolo.com/>
- [9] LangChain, Inc., *LangGraph – Build Stateful Multi-Actor Applications with LLMs*, 2024.
<https://langchain-ai.github.io/langgraph/>
- [10] OpenAI, *OpenAI API Reference*, 2024.
<https://platform.openai.com/docs/api-reference>
- [11] OpenAI, *GPT-4o – Multimodal Model*, 2024.
<https://openai.com/index/hello-gpt-4o/>
- [12] Meta Platforms, Inc., *React – A JavaScript Library for Building User Interfaces*, 2024.
<https://react.dev/>
- [13] Vercel, Inc., *Next.js – The React Framework for the Web*, 2024.
<https://nextjs.org/>
- [14] The PostgreSQL Global Development Group, *PostgreSQL: The World's Most Advanced Open Source Relational Database*, 2024.
<https://www.postgresql.org/>
- [15] A. Kiefer, *pgvector – Open-source Vector Similarity Search for Postgres*, 2024.
<https://github.com/pgvector/pgvector>
- [16] Redis Ltd., *Redis – The Real-time Data Platform*, 2024.
<https://redis.io/>

- [17] MinIO, Inc., *MinIO – High Performance Object Storage*, 2024.
<https://min.io/>
- [18] H. Krekel et al., *pytest: Simple Powerful Testing with Python*, 2024.
<https://docs.pytest.org/>
- [19] Microsoft, *Playwright – Fast and Reliable End-to-End Testing for Modern Web Apps*, 2024.
<https://playwright.dev/>
- [20] Artifex Software, Inc., *PyMuPDF (fitz) – Python Bindings for MuPDF*, 2024.
<https://pymupdf.readthedocs.io/>
- [21] S. Canny, *python-docx – Create and Modify Word Documents*, 2024.
<https://python-docx.readthedocs.io/>
- [22] R. Smith, *An Overview of the Tesseract OCR Engine*, Proc. ICDAR, pp. 629–633, 2007.
<https://github.com/tesseract-ocr/tesseract>
- [23] Amazon Web Services, *Amazon Textract – Extract Text and Data from Documents*, 2024.
<https://aws.amazon.com/textract/>
- [24] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS, 2020.
<https://arxiv.org/abs/2005.11401>
- [25] T. Guo et al., *Large Language Model Based Multi-Agents: A Survey of Progress and Challenges*, arXiv:2402.01680, 2024.
<https://arxiv.org/abs/2402.01680>
- [26] W. X. Zhao et al., *A Survey of Large Language Models*, arXiv:2303.18223, 2023.
<https://arxiv.org/abs/2303.18223>
- [27] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
<https://continuousdelivery.com/>
- [28] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed., O'Reilly Media, 2021.
<https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/>